Analisi Tecnica del Firmware

Progetto *Vera-Module-RBL-C64*

Analisi del progetto a cura di RetroBit Lab

21 ottobre 2026

Indice

1 Introduzione **1**

2 Architettura del Sistema **1**
2.1 Componenti Chiave 1
2.2 Mappa I/O 1
2.3 Ambiente di build PlatformIO 2
2.4 Interfacciamento Bus 2
2.5 Ripartizione dei core 3

3 Mapping dei Pin (ESP32-S3FN8, QFN56) **3**
3.1 Pin di strapping — comportamento al boot con il C64 collegato 4
3.2 Quarzo oscillatore 40 MHz (pin 53–54) 5

4 Analisi delle Tempistiche **5**
4.1 Sequenza di un ciclo 6
4.2 Percorso critico 6
4.3 Accesso diretto ai registri GPIO 6
4.4 Diagramma temporale di una lettura GeoRAM 7

5 Frammento di Codice Sorgente **7**

6 Diagramma di Flusso del Firmware **9**

7 Verifiche hardware **9**

8 Conclusione **10**

1 Introduzione

Questo documento presenta un'analisi tecnica del firmware ESP32 per il progetto *Vera-Module-RBL-C64*. Il firmware, eseguito su un ESP32-S3, si interfaccia con il bus del processore 6510 attraverso la **porta di espansione del Commodore 64**. Espone i registri della scheda video VERA (FPGA) e una **GeoRAM da 256 KB** emulata nella RAM interna dell'ESP32.

Il progetto deriva dalla versione Atari (*Vera-Module-RBL-XE*): schema e PCB sono ancora quelli Atari, con le modifiche hardware elencate in `PIN-MAPPING.md` §9. L'analisi si concentra sulla fattibilità temporale dell'interazione fra la MCU e la CPU, sulla struttura del loop del bus e sulla ripartizione del lavoro fra i due core.

2 Architettura del Sistema

2.1 Componenti Chiave

- **CPU 6510 (MOS):** il processore del C64, a **0.985 MHz** (PAL) o **1.023 MHz** (NTSC). PHI2 alto dura ≈490 ns.
- **MCU: ESP32-S3FN8** (QFN56, 7×7 mm) — dual-core Xtensa LX7 @ 240 MHz, 512 KB SRAM, 8 MB flash in-package Quad SPI, 45 GPIO.
- **FPGA VERA:** iCE40UP5K con il firmware VERA; risponde ai registri $DF00–$DF1F quando l'ESP32 asserisce `DEV_SEL_N`.

2.2 Mappa I/O

La scheda occupa le due aree I/O della porta di espansione, `IO1` ($DE00–$DEFF) e `IO2` ($DF00–$DFFF). Sul C64 queste aree non hanno memoria interna: non serve disabilitare nulla sulla scheda madre, basta pilotare il bus dati.

Indirizzo	Accesso	Funzione	Chi risponde
$DE00–$DEFF	R/W	Finestra GeoRAM (pagina da 256 B)	ESP32, `georam_256k[]`
$DF00–$DF1F	R/W	Registri VERA	FPGA, `DEV_SEL_N` asserito
$DFFD	W	Latch selezione scheda	ESP32 (snoop)
$DFFE	W	GeoRAM, registro pagina (0–63)	ESP32 (snoop)
$DFFF	W	GeoRAM, registro blocco (0–15)	ESP32 (snoop)

VERA e GeoRAM rispondono **solo a scheda selezionata**: un valore ≠ 0 scritto in $DFFD seleziona, `0` deseleziona. Il latch risponde sempre; con la scheda deselezionata le scritture a $DFFE/$DFFF sono ignorate.

La GeoRAM è organizzata in 16 blocchi da 16 KB, ognuno di 64 pagine da 256 byte:

$$\text{offset} = \text{blocco} \times 16384 + \text{pagina} \times 256 + (\text{addr} \;\&\; \$FF)$$

I registri sono a sola scrittura, si azzerano all'accensione e sopravvivono al reset del C64, come nell'hardware originale. Sono decodificati esattamente a $DFFE/$DFFF, senza i mirror $DF80–$DFFD di alcune GeoRAM: per questo il latch a $DFFD non va in conflitto.

2.3 Ambiente di build PlatformIO

Il file `platformio.ini` definisce un solo ambiente, `esp32s3fn8`. Opzioni di compilazione (`-D`):

Macro	Default	Effetto
`CARD_SEL_LATCH`	`0xFD`	offset in `IO2` del latch di selezione ($DFFD)
`IO_QUALIFY_NS`	200	attesa massima di `/IO1`/`IO2` dopo il fronte di salita di PHI2
`VERA_TRACE_REGS`	0	log di ogni accesso ai registri VERA (solo debug)
`VERA_POST_CDONE_DELAY_US`	1000	attesa fra `CDONE` e il rilascio del reset del C64

2.4 Interfacciamento Bus

Ogni ciclo è qualificato dalle linee `/IO1` e `/IO2` della porta di espansione. Nel C64 le genera il 74LS139 a valle del PLA, che le abilita solo con `AEC` alto, cioè solo per gli accessi della CPU: i fetch del VIC-II, i cui indirizzi compaiono anch'essi sulla porta, non le asseriscono e vengono ignorati. `/IO1` e `/IO2` decodificano già A8–A15, quindi al firmware servono solo A0–A7 come offset nella pagina.

L'adattamento di livello 5 V ↔ 3.3 V è affidato a cinque **74LVC4245** (direzione cablata, tranne `U19`) e a due porte **SN74HCT04** (`U23`). I riferimenti `U*` seguono la netlist Atari del 2026-09-01; dettaglio in `PIN-MAPPING.md` §6:

- **U19 — Bus dati D0–D7 (GPIO 4–11):** bidirezionale, `DIR` pilotato da `U23` con R/W_ e PHI2. L'ESP32 abilita le proprie uscite con `GPIO.enable_w1ts` solo nelle letture della finestra GeoRAM. Il gating di `~OE_` va modificato in `NAND(PHI2, NAND(R/W, AND(DEV_SEL_N, /IO1)))`: attivo in scrittura, nelle letture VERA e nelle letture GeoRAM.
- **U4 — A0–A7 (GPIO 12–19)** e **U3 — A8–A15:** ingressi C64→ESP32. A8–A15 sono cablati ma non usati dalla decodifica. GPIO 19/20 condividono i pad USB; USB CDC è disabilitato.
- **U6 — PHI2, R/W_, /IO1, /IO2 (GPIO 1, 2, 41, 42):** ingressi C64→scheda. GPIO 41 e 42 erano `EXTSEL_N` e `MPD` sulla versione Atari e vanno ricablati (`PIN-MAPPING.md` §9).
- **U16 — ARESET (GPIO 37) e IRQ della VERA:** uscite scheda→C64; entrambe devono essere open-drain verso le linee wired-OR `/RESET` e `/IRQ`.
- **Connessioni dirette (3.3 V):** `DEV_SEL_N` (GPIO 40) → /CS della VERA; `CRESET` (GPIO 38) → reset VERA; `CDONE` (GPIO 39) ← VERA.

PHI2, /IO1 e /IO2 sono letti con il **Dedicated GPIO input bundle** (ee.get_gpio_in, 1 ciclo CPU); DEV_SEL_N è l'unico canale del bundle di uscita (ee.wr_mask_gpio_out). I bundle sono creati dal task del bus sul Core 0, che è il core che li usa.

2.5 Ripartizione dei core

Core	Cosa gira	Note
0	solo MonitorTask (loop del bus)	priorità configMAX_PRIORITIES - 1, non cede mai la CPU; l'idle task del Core 0 è tolto dal task watchdog (disableCore0WDT())
1	setup(), loop(), Serial	loop() svuota log_ring e stampa; qui gira anche l'interrupt della UART

Il loop del bus passa gli eventi (cambio del latch; accessi ai registri VERA con VERA_TRACE_REGS=1) a loop() tramite log_ring, un ring buffer *single-producer/single-consumer* senza lock. L'inserimento costa poche istruzioni; il timestamp è il tick FreeRTOS in millisecondi. Se il ring è pieno l'evento viene scartato e loop() stampa quanti eventi sono andati persi.

La versione precedente usava xQueueSend() + esp_timer_get_time() (spinlock e sezione critica, ≈1 µs) dentro il loop del bus, e faceva girare il loop sul Core 1, lo stesso core di loop(): il task del bus, a priorità massima e senza mai cedere la CPU, impediva a loop() di stampare il log.

3 Mapping dei Pin (ESP32-S3FN8, QFN56)

I pin fisici del package QFN56 sono numerati 1–56 (più il pad GND centrale). La tabella riporta i segnali con il numero GPIO logico **e** il numero di pad fisico (Table 2-1 del datasheet ESP32-S3 v2.2).

Nota importante: il MCU *richiede* un **quarzo esterno a 40 MHz** sui pin dedicati **XTAL_P** (pin 54) e **XTAL_N** (pin 53); senza quarzo il chip non si avvia (sezione 3.2).

Segnale	GPIO	Pin QFN56	IO MUX	Dir.	Note
Bus dati D0–D7 — bidirezionale via 74LVC4245 U19 (Bank-0 bit 4–11)					
D0–D7	4–11	9–16	GPIO4–11	I/O	D0 ha un glitch LOW di 60 µs al power-up
Bus indirizzi A0–A7 — input via 74LVC4245 U4 (Bank-0 bit 12–19)					
A0–A2	12–14	17–19	GPIO12–14	IN	
A3	15	21	XTAL_32K_P	IN	pad RTC usato come GPIO; pin 20 = VDD3P3_RTC
A4	16	22	XTAL_32K_N	IN	pad RTC usato come GPIO
A5, A6	17, 18	23, 24	GPIO17, 18	IN	
A7	19	25	USB_D−	IN	USB CDC disabilitato
Bus indirizzi A8–A15 — input via U3, cablati ma non usati dalla decodifica					
A8, A9	20, 21	26, 27	USB_D+, GPIO21	IN	
A10, A11	35, 36	40, 41	GPIO35, 36	IN	
A12, A13	33, 34	38, 39	GPIO33, 34	IN	
A14, A15	45, 46	51, 52	GPIO45, 46	IN	pin di strapping (sezione 3.1)
Controllo — U6 (ingressi) e U16 (uscite)					
PHI2	1	6	GPIO1	IN	Dedicated GPIO input ch 0; 0.985/1.023 MHz
R/W_	2	7	GPIO2	IN	HIGH = lettura
/IO1	41	47	MTDI	IN	Dedicated GPIO input ch 1; ex `EXTSEL_N`, ex-JTAG
/IO2	42	48	MTMS	IN	Dedicated GPIO input ch 2; ex `MPD`, ex-JTAG
ARESET	37	42	GPIO37	OUT	`/RESET` del C64 (LOW asserito)
Segnali diretti (senza level shifter, 3.3 V)					
DEV_SEL_N	40	45	MTDO	OUT	/CS VERA; Dedicated GPIO output ch 0; ex-JTAG
CRESET	38	43	GPIO38	OUT	/RESET VERA
CDONE	39	44	MTCK	IN	VERA configurata (HIGH = OK); ex-JTAG
—	3	8	GPIO3	—	non usato; resistenza 10 kΩ obbligatoria (strapping)
BOOT0	0	5	GPIO0	IN	pulsante BOOT / auto-reset; non usato dal firmware
Debug					
UART TX	43	49	U0TXD	OUT	Serial debug @ 115200 baud
UART RX	44	50	U0RXD	IN	non utilizzato dal firmware

Attenzione: i pin QFN56 30–35 (GPIO 27–32) sono collegati alla flash Quad SPI in-package (variante FN8) e non devono mai essere connessi esternamente; il pin 28 (GPIO 26, `SPICS1`) è il chip select della PSRAM in-package, assente sulla FN8, e resta sconsigliato (datasheet Table 2-14). GPIO 47 (pin 37) e GPIO 48 (pin 36) sono liberi. I pin 53 e 54 sono pad analogici dedicati al quarzo da 40 MHz.

3.1 Pin di strapping — comportamento al boot con il C64 collegato

L'ESP32-S3 campiona quattro pin di strapping durante la finestra di reset al power-on, *prima che qualsiasi codice venga eseguito*. `PIN_ARESET` (GPIO 37) diventa un'uscita solo dentro `setup()`: al momento del campionamento l'ESP32 non ha ancora asserito il reset del C64. È il circuito di power-on reset del C64 (timer 556) a tenere basso il `RESET#` del 6510, mettendo il bus indirizzi in alta impedenza. I 74LVC4245, con gli ingressi lato C64 flottanti, *non* garantiscono uscite in alta impedenza: per A14 e A15 il livello campionato dipende dal pull-down interno dell'ESP32 contro un nodo non pilotato. **Da verificare sulla scheda**; se necessario aggiungere un pull-down da 10 kΩ lato ESP32 su GPIO 45 e GPIO 46.

- **GPIO 0 (pin 5) — boot mode, pull-up interno ≈45 kΩ.** Rete `BOOT0` (`R33` 4.7 kΩ verso 3V3, pulsante `SW3`, `C61`, `R54` verso `Q4`); non collegato al C64. Campionato HIGH → boot SPI dalla flash integrata.

- **GPIO 3 (pin 8) — sorgente JTAG, nessun pull interno.** Non usato dal firmware; resta la resistenza esterna da 10 kΩ (ex `RAMBO_EN`), obbligatoria perché il datasheet vieta l'alta impedenza durante lo strapping. Con gli eFuse di default il livello è comunque ignorato.
- **GPIO 45 (pin 51) — tensione VDD_SPI, pull-down interno ≈5 kΩ.** Collegato ad A14 tramite `U3`. Campionato LOW → VDD_SPI = 3.3 V da VDD3P3_RTC (Table 3-4, eFuse di default), il valore richiesto dalla flash in-package della FN8 (Table 1-1). Con A14 HIGH al campionamento VDD_SPI sarebbe 1.8 V e il chip non si avvierebbe: A14 deve essere LOW al power-on (da verificare), oppure si programmano gli eFuse `EFUSE_VDD_SPI_FORCE` = 1 e `EFUSE_VDD_SPI_TIEH` = 1 (irreversibile) per fissare 3.3 V.
- **GPIO 46 (pin 52) — messaggi ROM su UART0, pull-down interno ≈5 kΩ.** Collegato ad A15 tramite `U3`. Campionato LOW → messaggi ROM del primo stage disabilitati, corretto per la produzione.

3.2 Quarzo oscillatore 40 MHz (pin 53–54)

L'ESP32-S3FN8 richiede un quarzo esterno da **40 MHz** per il riferimento del PLL che alimenta la CPU a 240 MHz, l'interfaccia flash e i divisori UART. I pin sono analogici dedicati, non condivisi con GPIO.

```
XTAL_P (54) --[C1 10pF]-- GND
      |
   [X1 40MHz]     <- Rs 0-33 Ohm in serie (opzionale)
      |
XTAL_N (53) --[C2 10pF]-- GND
```

Componente	Valore	Note
X1 – quarzo	40 MHz, SMD 3225 o 2016	±10 ppm; verificare C_L dal datasheet
C1, C2 – condensatori di carico	10 pF, 0402, ±5 %	$C_{ext} = 2C_L - C_{stray}$; $C_{stray} \approx 3$–5 pF
Rs – resistenza serie	0–33 Ω (opzionale)	Limita il drive dell'oscillatore

Regole di layout PCB: tracce XTAL_P/N più corte possibile (<5 mm); condensatori di carico ai pad del chip; piano di massa solido sotto il quarzo; isolamento dalle tracce PHI2 e D0–D7.

4 Analisi delle Tempistiche

- **Ciclo 6510:** $T_{6510} \approx 1015$ ns (PAL, 0.985 MHz) o ≈ 978 ns (NTSC, 1.023 MHz).
- **Finestra PHI2 alto:** $T_{PHI2\uparrow} \approx 490$ ns. Nella fase PHI1 il bus appartiene al VIC-II.
- **Ciclo ESP32 (240 MHz):** $T_{ESP32} \approx 4.17$ ns, quindi

$$\frac{T_{PHI2\uparrow}}{T_{ESP32}} = \frac{490\,\text{ns}}{4.17\,\text{ns}} \approx 117 \text{ cicli ESP32.}$$

4.1 Sequenza di un ciclo

1. **Fronte di salita di PHI2**, letto con `ee.get_gpio_in`.
2. **Qualificazione**: attesa limitata (`IO_QUALIFY_NS`, default 200 ns) di `/IO1` o `/IO2` basso. Se nessuna delle due si attiva, il ciclo non è della scheda (altro indirizzo o fetch VIC-II) e viene saltato. Un ciclo `/IO1` a scheda deselezionata viene scartato subito, senza leggere il bus.
3. **Lettura di `GPIO.in`** (A0–A7, R/W_): a questo punto `AEC` è alto e l'indirizzo è quello del 6510.
4. **`/IO1`, finestra GeoRAM**: in lettura l'ESP32 pilota D0–D7 con `win[off8]`, dove `win` è il puntatore alla pagina corrente, tenuto in un registro e ricalcolato solo quando si scrive $DFFE/$DFFF.
5. **`/IO2`**: per $DF00–$DF1F a scheda selezionata asserisce `DEV_SEL_N` (risponde la FPGA).
6. **Scritture**: il dato del 6510 a 1 MHz è valido fino a ≈200 ns dopo la salita di PHI2. Il firmware rilegge `GPIO.in` finché PHI2 è alto (PHI2 è nello stesso registro) e usa l'ultimo campione; poi, durante PHI1, aggiorna GeoRAM, latch e registri.
7. **Fronte di discesa di PHI2**: bus rilasciato e `DEV_SEL_N` de-asserito.

4.2 Percorso critico

Conteggio dal disassemblato della ELF (`xtensa-esp32s3-elf-objdump`), a partire dalla lettura di `GPIO.in`:

Fino a	Versione precedente	Versione corrente
`DEV_SEL_N` asserito (registri VERA)	≈23 istruzioni	≈9 istruzioni
dato GeoRAM pilotato su D0–D7	≈40 istruzioni	≈18 istruzioni

La versione precedente ricalcolava a ogni ciclo l'offset GeoRAM da pagina e blocco letti dalla IRAM e trattava `/IO1` e `/IO2` nello stesso ramo. Ora i dati del loop (`lut_drive`, registri GeoRAM, `log_ring`) sono in DRAM e il codice in IRAM; la GeoRAM da 256 KB resta in `.iram0.bss`. Con ≈117 cicli disponibili il margine c'è anche considerando l'attesa di `/IOx`; i tempi reali (letture APB, sincronizzatori GPIO, ritardo di `/IOx`) vanno confermati con un analizzatore logico.

4.3 Accesso diretto ai registri GPIO

- `GPIO.in` — i 32 GPIO del bank-0 in una lettura: PHI2, R/W_, D0–D7, A0–A7.
- `GPIO.out` + `GPIO.enable_w1ts` — pilotaggio del bus dati con la LUT `lut_drive[]`; `GPIO.enable_w1tc` lo rilascia.
- `GPIO.out1_w1ts` — rilascio di `CRESET` e `ARESET` all'avvio.

4.4 Diagramma temporale di una lettura GeoRAM

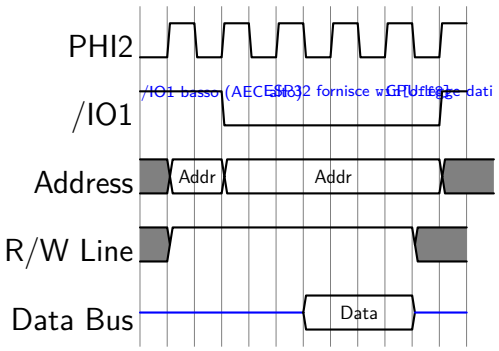


Figura 1: Ciclo di lettura della finestra GeoRAM gestito dall'ESP32.

5 Frammento di Codice Sorgente

Estratto di `main.cpp`: le definizioni del Dedicated GPIO e il nucleo del `MonitorTask` (setup dei bundle e sequenza di reset omessi).

```
#define DEDIC_IN_PHI2         (1u << 0)   /* GPIO 1           */
#define DEDIC_IN_IO1N         (1u << 1)   /* GPIO 41, active LOW */
#define DEDIC_IN_IO2N         (1u << 2)   /* GPIO 42, active LOW */
#define DEDIC_IN_IO_MASK      (DEDIC_IN_IO1N | DEDIC_IN_IO2N)
#define DEDIC_OUT_DEVSELN     (1u << 0)   /* GPIO 40, active LOW */

/* Base of the 256-byte GeoRAM page currently visible at $DE00. */
static inline uint8_t * IRAM_ATTR georam_window(uint8_t block, uint8_t
    page)
{
     return &georam_256k[((uint32_t)block << 14) | ((uint32_t)page << 8)
    ];
}
```

Listing 1: Canali Dedicated GPIO e helper

```
bool      selected = false;
uint8_t  page = 0, block = 0;
uint8_t *win  = georam_window(block, page);

for (;;) {
    /* 1. Fronte di salita di PHI2 (1 istruzione TIE) */
    while (!(cpu_ll_read_dedic_gpio_in() & DEDIC_IN_PHI2));

    /* 2. Qualificazione: attesa limitata di /IO1 o /IO2 */
    uint32_t io_n, t0 = cpu_ll_get_cycle_count();
    do {
        io_n = ~cpu_ll_read_dedic_gpio_in() & DEDIC_IN_IO_MASK;
    } while (!io_n && cpu_ll_get_cycle_count() - t0 < IO_QUALIFY_CYCLES)
    ;
    if (!io_n || ((io_n & DEDIC_IN_IO1N) && !selected)) {
        while (cpu_ll_read_dedic_gpio_in() & DEDIC_IN_PHI2);
```

```
        continue;
    }

    uint32_t g_lo = GPIO.in;
    uint8_t  off8 = decode_addr_lo(g_lo);

    if (io_n & DEDIC_IN_IO1N) {             /* 3a. GeoRAM $DE00-$DEFF */
        if (g_lo & (1UL << PIN_RW)) {
            bus_drive(win[off8]);
            while (cpu_ll_read_dedic_gpio_in() & DEDIC_IN_PHI2);
            bus_release();
        } else {                            /* ultimo campione a PHI2
    alto */
            uint32_t last = g_lo, now;
            while ((now = GPIO.in) & (1UL << PIN_PHI2)) last = now;
            win[off8] = decode_data(last);
        }
        continue;
    }

    /* 3b. IO2 $DF00-$DFFF */
    bool is_vera = selected && off8 < VERA_REG_COUNT;
    if (is_vera)
        cpu_ll_write_dedic_gpio_mask(DEDIC_OUT_CTRL_MASK, ~
    DEDIC_OUT_DEVSELN);
    if (g_lo & (1UL << PIN_RW)) {
        while (cpu_ll_read_dedic_gpio_in() & DEDIC_IN_PHI2);
        cpu_ll_write_dedic_gpio_mask(DEDIC_OUT_CTRL_MASK,
    DEDIC_OUT_CTRL_IDLE);
        continue;
    }
    uint32_t last = g_lo, now;
    while ((now = GPIO.in) & (1UL << PIN_PHI2)) last = now;
    cpu_ll_write_dedic_gpio_mask(DEDIC_OUT_CTRL_MASK,
    DEDIC_OUT_CTRL_IDLE);
    uint8_t data = decode_data(last);

    /* 4. Effetti durante PHI1 */
    if (off8 == CARD_SEL_LATCH) {
        selected = (data != 0);              /* + log_send() al Core 1
    */
    } else if (selected && off8 == GEORAM_PAGE_REG) {
        page = data & GEORAM_PAGE_MASK;   win = georam_window(block,
    page);
    } else if (selected && off8 == GEORAM_BLOCK_REG) {
        block = data & GEORAM_BLOCK_MASK; win = georam_window(block,
    page);
    }
}
```

Listing 2: Nucleo del MonitorTask (Core 0)

6 Diagramma di Flusso del Firmware

Il `MonitorTask` gira da solo sul Core 0 con priorità massima; il Core 1 stampa gli eventi del `log_ring` (sezione 2.5).

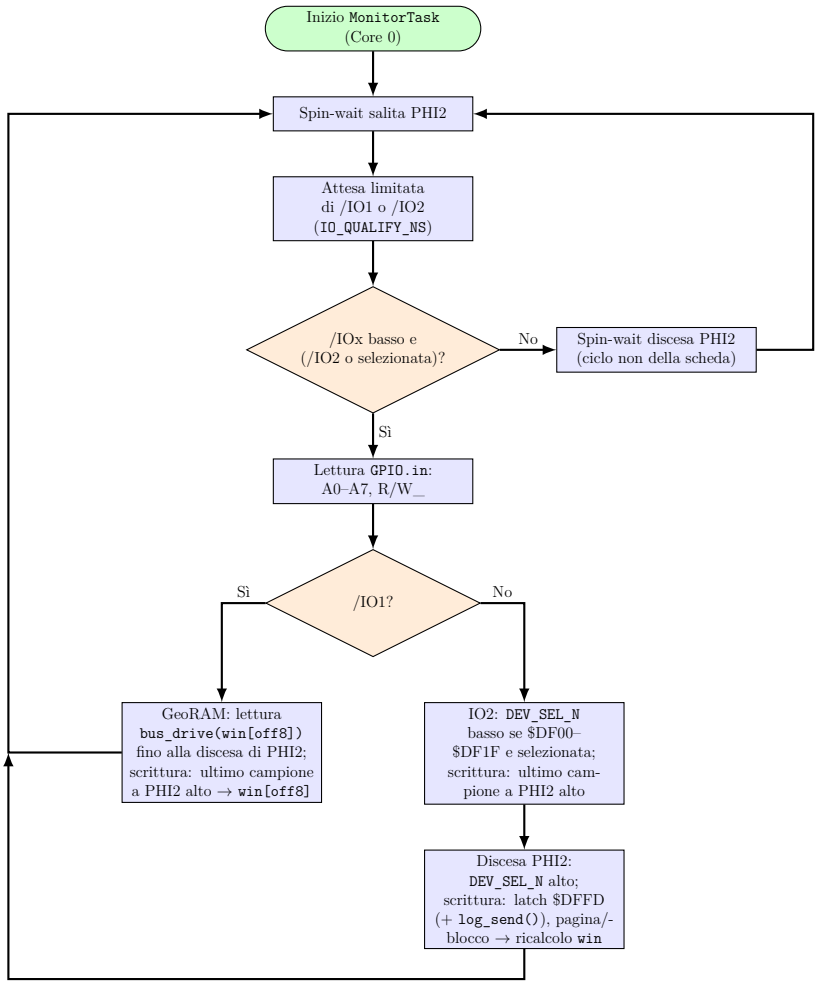


Figura 2: Diagramma di flusso semplificato del `MonitorTask` (ESP32-S3FN8, C64).

7 Verifiche hardware

I punti seguenti derivano dalla netlist della versione Atari e dalle modifiche previste per il C64 (`PIN-MAPPING.md` §9); vanno confermati sullo schema aggiornato e con l'oscilloscopio. I requisiti della VERA sul bus (RD_n e WR_n distinti, hold del dato dopo la salita di WR_n, effetti collaterali alla fine di RD_n, riconfigurazione con CTRL bit 7) sono descritti in `VERA-ATARI-HW-REQUIREMENTS.md` e valgono anche per il C64.

- **Ritardo di `/IO1`//`IO2`.** `IO_QUALIFY_NS` va tarato misurando il ritardo di `/IOx` rispetto a PHI2 (PLA + 74LS139, qualche decina di ns): troppo corto e la scheda perde cicli, troppo lungo e resta meno tempo per il dato in lettura.
- **Gating di `U19`.** `~OE_` deve essere attivo solo in scrittura, nelle letture VERA e nelle letture GeoRAM: `NAND(PHI2, NAND(R/W, AND(DEV_SEL_N, /IO1)))`. Con la

scheda deselezionata una lettura a $DExx abilita comunque U19 senza che l'ESP32 piloti il bus: su un C64 senza cartuccia quegli indirizzi sono già *open bus*, quindi il comportamento è equivalente.

- **Hold del dato in scrittura verso la VERA.** Lo stesso fronte di discesa di PHI2 chiude WR_n e U19: verificare che il dato resti stabile dopo la salita di WR_n.
- **/RESET e /IRQ open-drain.** Sul C64 sono linee wired-OR: ARESET e l'IRQ della VERA (oggi push-pull) richiedono uno stadio open-drain (per esempio 74LVC1G07).
- **Linee non usate.** /GAME, /EXROM, /ROML, /ROMH, BA, /DMA, /NMI e Dot Clock restano scollegate: la scheda non mappa ROM e non fa DMA.

Sequenza di reset e configurazione

setup() (Core 1) tiene ARESET e CRESET bassi prima ancora di Serial.begin(), crea il MonitorTask sul Core 0 e rilascia il controllo. Il task rilascia CRESET, attende CDONE (timeout 5 s), aspetta VERA_POST_CDONE_DELAY_US (default 1000 µs) per il POR interno della VERA e solo allora rilascia ARESET. La sequenza vale solo al power-on; il tasto di reset del C64 deve continuare a funzionare, quindi ARESET va realizzato open-drain.

8 Conclusione

Con PHI2 alto per ≈490 ns l'ESP32-S3 a 240 MHz dispone di circa 117 cicli per ciclo di bus, contro i 67 della versione Atari. La qualificazione tramite /IO1 e /IO2 riduce la decodifica alla sola lettura di A0–A7, e il percorso critico dopo la lettura di GPIO.in è di circa 9 istruzioni fino a DEV_SEL_N e circa 18 fino al dato GeoRAM sul bus.

La ripartizione dei core tiene il Core 0 dedicato al bus, con il log passato al Core 1 tramite un ring buffer senza lock, e il campionamento delle scritture all'ultimo istante con PHI2 alto rende la cattura del dato indipendente dal ritardo con cui il 6510 lo presenta. Restano da misurare sulla scheda il ritardo di /IOx, il tempo dal fronte di PHI2 al dato GeoRAM e il comportamento di U19 dopo la modifica del gating.